

SC-Net Update — 22 April 2026

1. Optimisation Code Changes (Phase 26)

Background

The dual-task contrastive loss is defined in the paper (Eq. 7) as:

$$L_{dc} = L_{od}(C(\hat{y}_{sc}), \hat{y}_{od}) + L_{sc}(C^{-1}(\hat{y}_{od}), \hat{y}_{sc})$$

$C^{-1}(\hat{y}_{od})$ transforms OD predictions into sampling-point classification pseudo-labels. By definition it is a pure function of model predictions — no ground truth involved. The previous implementation violated this: it ran Hungarian matching between OD predictions and ground-truth targets first, then used only the matched query subset as pseudo-label candidates. Ground truth was hidden inside what should have been a prediction-only operation.

This update removes that dependency entirely.

Change 1 — `_get_sampling_point_classification_targets` (`optimization.py`)

Old signature: (`self`, `od_outputs`, `od_targets`) — GT required.

New signature: (`self`, `od_outputs`) — GT-free.

Old behaviour: Run Hungarian matching between OD predictions and GT targets → take only matched queries → convert matched predictions to SC point labels.

New behaviour:

```
For each batch item:
1. Softmax all Q OD queries
2. Keep only foreground queries (argmax class ≠ no-object)
   AND above confidence_threshold
3. For each of the seq_length sampling points:
   Winner-takes-all: the highest-confidence query whose
   predicted box covers this point assigns the label
4. Label shift: OD class k → SC label k+1 (background = 0)
```

Winner-takes-all makes the conversion deterministic — when multiple queries overlap the same point with different predicted classes, the one with higher softmax confidence wins rather than the last writer in iteration order.

The rounding convention (`round(x / interval) - 1`, clamped to `[0, seq_length-1]`) exactly matches the existing `od2sc_targets` function, so the label placement geometry is consistent across both directions of `L_dc`.

Change 2 — `dual_task_contrastive_loss.forward`

Old: `forward(self, od_outputs, sc_outputs, od_targets)` — GT required, always passed.

New: `forward(self, od_outputs, sc_outputs, od_targets=None)` — GT optional.

The hard-label path (default) calls

`_get_sampling_point_classification_targets(od_detached)` with no GT. The soft-label path (`use_soft_labels=True`, enabled by `--soft_dc`) retains the optional `od_targets` argument for backward compatibility — that path still needs GT to run Hungarian matching for the KL-divergence computation.

Change 3 — `set_dc_confidence` added to both loss classes

```
# dual_task_contrastive_loss
def set_dc_confidence(self, threshold: float) -> None:
    self.confidence_threshold = threshold

# spatio_temporal_contrast_loss (delegates down)
def set_dc_confidence(self, threshold: float) -> None:
    self.dc_loss.set_dc_confidence(threshold)
```

This gives the trainer a handle to update the confidence gate each epoch, enabling the annealing schedule described below.

Change 4 — `spatio_temporal_contrast_loss.forward`

Old:

```
od_targets_dc = [{k: v.clone() ...} for t in od_targets] # deep-copy for
dc_loss
dc_loss_val = self.dc_loss(od_outputs, sc_outputs, od_targets_dc) *
self.dc_weight
```

New:

```
# od_targets_dc deep-copy removed — dc_loss no longer receives GT
dc_loss_val = self.dc_loss(od_outputs, sc_outputs) * self.dc_weight
```

The `od_targets_dc` deep-copy existed solely to protect GT tensors from in-place mutation inside `dc_loss`. Since `dc_loss` no longer touches GT, it is removed.

Change 5 — Curriculum Confidence Annealing (`train.py`)

A GT-free C^{-1} creates a new stability risk: early in training when OD predicts mostly no-object, pseudo-labels are nearly all background and the SC contrastive direction of L_{dc} carries no useful signal. Worse, the few foreground predictions that do appear are likely wrong, so they inject noise into the SC branch.

The solution: start with an aggressive confidence threshold (default `dc_confidence_start=0.7`) so only highly certain OD predictions become pseudo-labels. Anneal it linearly down to `dc_confidence_threshold` (the floor) over the `dc_warmup_ramp` window — the same period over which `dc_weight` ramps from 0 to `delta`. Both the loss magnitude and the pseudo-label quality therefore open up together.

Schedule (implemented in `Trainer._compute_dc_confidence`):

Epoch range	Threshold
<code>< dc_warmup_hold</code>	<code>dc_confidence_start</code> (hold at ceiling)
<code>dc_warmup_hold</code> to <code>hold + ramp</code>	Linear interpolation from start $\rightarrow$ floor
<code>$\geq$ hold + ramp</code>	<code>dc_confidence_threshold</code> (floor, fixed)

New CLI argument: `--dc_confidence_start` (default `0.7`). Backward compatible: if `dc_warmup_hold=0` and `dc_warmup_ramp=0`, the threshold is fixed at `dc_confidence_threshold` from epoch 0.

`set_dc_confidence` is called in `train_one_epoch` alongside `set_dc_weight`. The current threshold is logged to TensorBoard (`Schedule/dc_confidence`) and printed in each epoch summary as `DC_conf: X.XXX`.

Test Coverage

21 new tests added in `tests/test_dc_loss.py`:

Group	Tests
GT-free output shape	1
No-object filtering	1
Confidence threshold exclusion / inclusion	2
Label shift (OD class $k \rightarrow$ SC label $k+1$)	2
Winner-takes-all deduplication	2
<code>set_dc_confidence</code> updates threshold	1
<code>set_dc_confidence</code> affects filtering	1
Delegation through <code>spatio_temporal_contrast_loss</code>	1
<code>forward</code> works without GT arg	1

Group	Tests
<code>forward</code> does not pass GT to <code>dc_loss</code>	1
Confidence annealing schedule math	8

All 21 pass. No regressions in pre-existing tests.

2. v14 Fine-tuning — Current Run

Configuration (`configs/finetune_v14.yaml`)

Parameter	Value	Notes
Pretrained backbone	<code>checkpoints_v14/best_model.pth</code>	Full 300-epoch pre-train
Peak LR	2.0e-5	Slightly lower than v13's 2.5e-5 — backbone fully converged
LR schedule	CosineWarmRestarts, T0=60	Same as v12 (strategy that drove F1 to 0.739)
Epochs	300	With patience=120 early stopping
DC hold / ramp / delta	20 / 40 / 0.5	Same as v12
<code>dc_confidence_start</code>	0.7	Phase 26 — anneals to 0.4 over ramp (epochs 20–60)
<code>dc_confidence_threshold</code>	0.4	Floor after ramp
<code>dc_temperature_start</code>	3.0	Anneals to 1.0 over ramp
SWA	On from epoch 100	Weight averaging for late-training stability
Focal loss	$\gamma=2.0$	SC branch class imbalance
<code>boost_nonsig</code>	True	2× weight on Non-significant stenosis class
Ordinal EMD loss	weight=0.5	Penalises severity ordering errors
Balanced sampling	True	Inverse-frequency sample weighting
Effective batch size	16	2 GPU × 2 samples × 4 accumulation steps

Key Differences from v13 Fine-tuning

	v13 fine-tuning	v14 fine-tuning
Pre-trained backbone	110/300 epochs (killed early)	300/300 epochs (fully converged)
SE fusion gates	Under-converged	Fully trained
Parallel 2D stream	Under-converged	Fully trained
C ⁻¹ in L_dc	GT-dependent (Hungarian)	GT-free + confidence annealing
Peak LR	2.5e-5	2.0e-5

3. Projected Results

Baseline to Beat

Metric	v12 fine-tuning
Stenosis ACC	0.736
Stenosis F1	0.739
Stenosis Precision	0.743
Stenosis Recall	0.736
Non-sig Recall	0.639
Sig Recall	0.733
Plaque F1	0.502

v13 fine-tuning regressed to F1=0.577 — below v12 — because pre-training was aborted at ep110. v14 is the retry with the full backbone. v12 is the correct comparison, not v7-ft.

Projected v14 Range

Stenosis F1: 0.76–0.84

The projection range reflects uncertainty across two independent improvements stacking:

Lower bound (0.76): If GT-free L_dc has minimal effect and the improvement comes only from the fully-converged backbone, a modest gain over v12's 0.739 is expected. The SE gates and truly-parallel 2D/3D streams add spatial feature diversity that v12 didn't have — conservative estimate puts this at +2–3% F1.

Upper bound (0.84): If GT-free L_dc meaningfully cleans up the SC pseudo-label signal during fine-tuning (particularly for the fresh 6-class heads in the first 60 epochs), combined with the better backbone, the mutual supervision loop becomes more self-consistent earlier. The DC temperature annealing (3.0→1.0) and confidence annealing (0.7→0.4) operating in sync should prevent the noisy early pseudo-labels that can lock in bad representations.

Non-sig Recall: Expected to hold at or above v12's 0.639. The `boost_nonsig` + constrained calibration combination that achieved Non-sig Recall=0.639 in v12 is retained. The GT-free L_dc may help if the old

Hungarian-matching step was inadvertently leaking GT label information that the model was exploiting rather than learning.

Plaque F1: Modest improvement expected (0.50–0.53). Plaque prediction is driven primarily by the spatial branch and calibration — the architecture improvements here are the same as what lifted v12 over v7-ft.

SC branch ACC: Should remain above 0.80. v8-ft demonstrated that focal_gamma=3.0 destabilises this branch; v14 keeps $\gamma=2.0$ (same as v12 which achieved SC ACC ~0.80+).

What to Watch For

1. **Epoch 20–25:** DC loss activates. Verify it contributes ~0.1–0.5 to total loss, not orders of magnitude larger. If it spikes and doesn't recover, the confidence gate may need raising for this dataset/backbone combination.
2. **Epoch 60 (LR restart, T0=60):** v12's F1 climbed from ~0.40 to 0.584 in the post-ep60 window. Expect the same inflection point — the second cosine cycle with DC fully active is where v14 should differentiate from v13.
3. **SWA from ep100:** Late-epoch averaged weights typically add +1–2% F1 over the raw best checkpoint. Important to evaluate the SWA model separately from `best_model.pth`.
4. **Calibration:** Run constrained 3D calibration (`--constrain_nonsig_recall 0.10`) — standard 2D calibration consistently collapses Non-sig recall to zero. The constrained search is mandatory to fairly evaluate Non-sig performance.

Post-run Evaluation Commands

```
# 1. Raw evaluation
python eval.py --model checkpoints_v14_finetune/best_model.pth \
  --pattern fine_tuning --data_root ./dataset/train

# 2. Constrained calibration
python calibrate.py --model checkpoints_v14_finetune/best_model.pth \
  --constrain_nonsig_recall 0.10 \
  --output calibration_thresholds_v14_constrained.json

# 3. Evaluate with calibration applied
python eval.py --model checkpoints_v14_finetune/best_model.pth \
  --calibration calibration_thresholds_v14_constrained.json \
  --use_constrained --pattern fine_tuning --data_root ./dataset/train

# 4. SWA model (if SWA was active)
python eval.py --model checkpoints_v14_finetune/swa_model.pth \
  --pattern fine_tuning --data_root ./dataset/train
```

4. Summary

Item	Status
Phase 26 implementation	Complete — committed b19aa49
21 new tests	All passing
v14 fine-tune config	Committed 316409e
v14 fine-tuning run	In progress — PID 3855461, logging to logs_finetune_v14.log
Next action	Await completion → eval → constrained calibration → compare vs v12